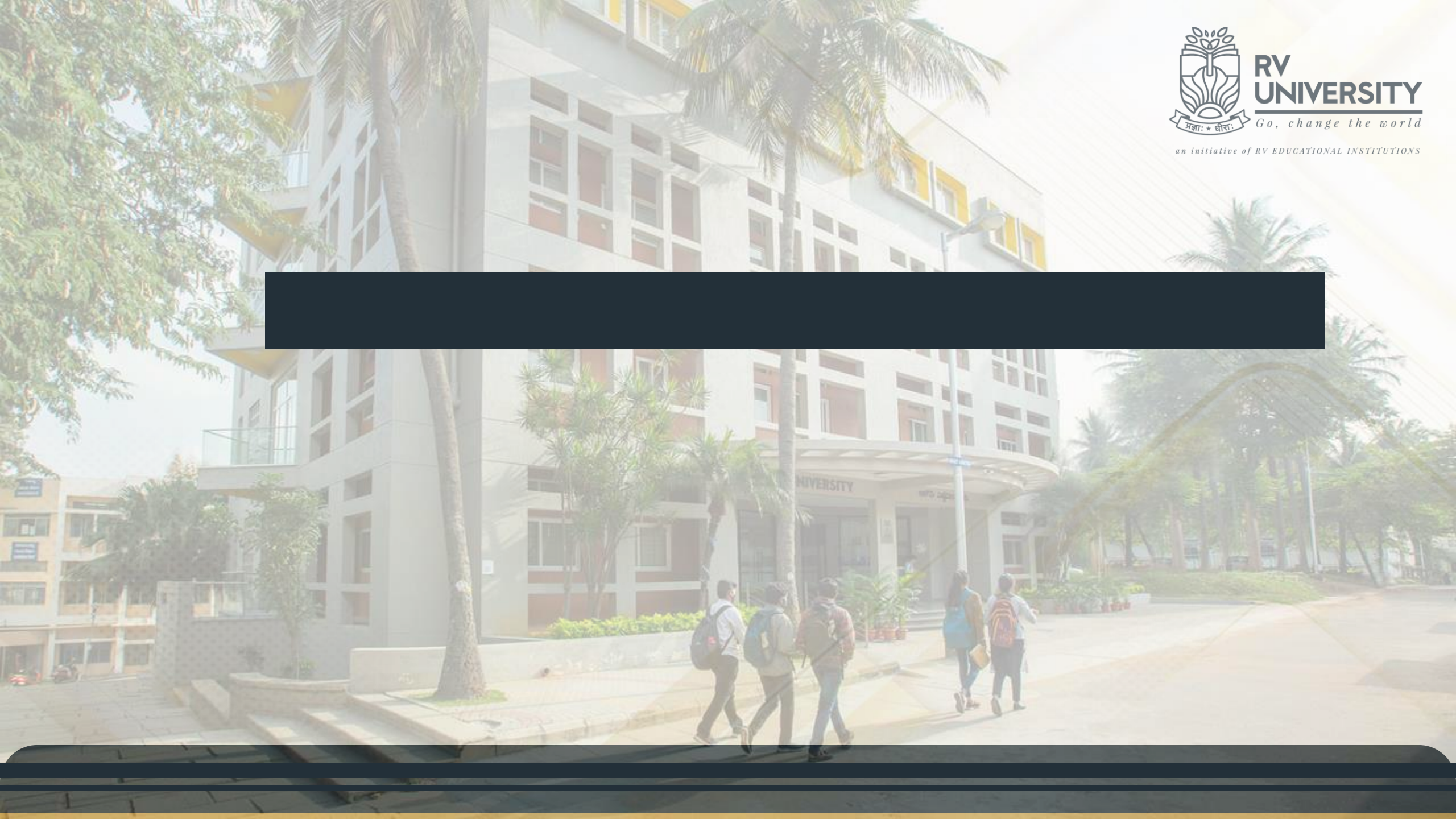


RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

UNIT 4: JavaScript: Functions, DOM, Forms, and Event Handlers

JavaScript is a high-level, interpreted programming language primarily used for building interactive and dynamic web pages.

JavaScript is mainly known for its client-side scripting capabilities. It runs directly in the web browser, allowing developers to create interactive and dynamic features that respond to user actions without requiring a page reload.

JavaScript is commonly used to handle events triggered by user actions, such as clicks, keypresses, and form submissions. Event listeners are used to execute specific code in response to these events.

The Document Object Model (DOM) represents the structure of an HTML document. JavaScript is used to manipulate the DOM, enabling dynamic updates to the content and structure of a web page.

While JavaScript is primarily associated with client-side scripting, Node.js allows developers to use JavaScript for server-side programming as well. This has led to the rise of full-stack JavaScript development.

Application of JavaScript

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Client-side validation
Dynamic drop-down menus
Displaying date and time
Displaying pop-up windows and dialog boxes (like an alert dialog box, confirm dialog box and prompt dialog box)
Displaying clocks etc.

Where to Place JavaScript Code

The <script> Tag
In HTML, JavaScript code is inserted between <script> and </script> tags.

Between the body tag of html
Between the head tag of html
In .js file (external javaScript)

Variable

- A variable is a name associated with a piece of data
- Variables allow you to store and manipulate data in your programs

```
var message  = "hi";
```

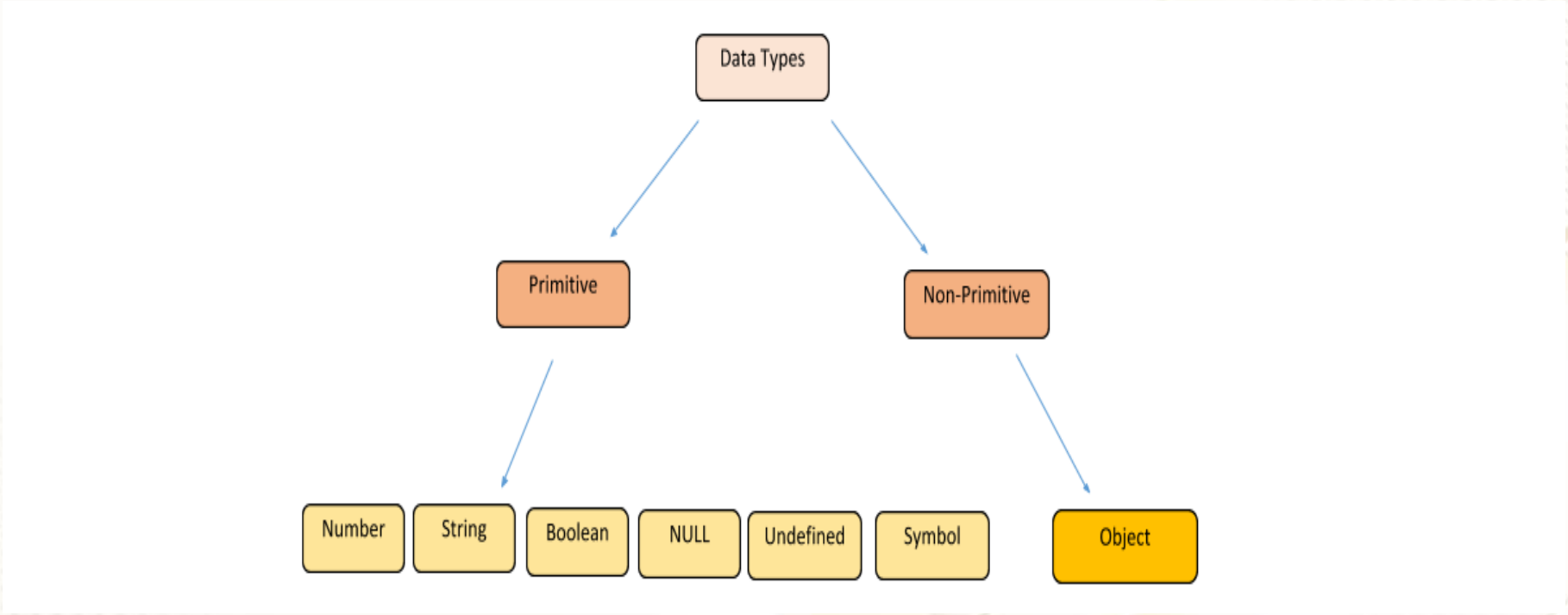
Variable definition should always start with ‘var’

No types are declared

JS is dynamically typed language

Same variable can hold different types during the life of the execution

Data Types


Data Types
Primitive
Non-Primitive
Number
String
Boolean
NULL
Undefined
Symbol
Object

Primitive Data Types

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Numbers - A number can be either an integer or a decimal
- Strings - A string is a sequence of letters or numbers enclosed in single or double quotes
- Boolean - True or False
- Undefined - A variable without a value, has the value undefined.
- Null -In JavaScript null is "nothing". In JavaScript, the data type of null is an object.

- var x1 = 34.00; // Written with decimals
  var x2 = 34; // Written without decimals
- var carName = "Volvo XC60"; // Using double quotes
  var carName = 'Volvo XC60'; // Using single quotes
- var y = 123e5; // 12300000
  var z = 123e-5; // 0.00123
- var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
- var car; // Value is undefined, type is undefined
- person = null; // Now value is null, but type is still an object

Implicit Data Types

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Although JavaScript does not have explicit data types, it does have implicit data types
- If you have an expression which combines two numbers, it will evaluate to a number
- If you have an expression which combines a string and a number, it will evaluate to a string

Difference Between Undefined and Null

- Undefined and null are equal in value but different in type:

```
typeof undefined          // undefined
typeof null               // object

null === undefined        // false
null == undefined         // true
```

- Example:

```
var x = 4;
var y = 11;
var z = “cat”;
var q = “17”;
Ans = x + y;
   Ans => 15
Ans = z + x;
   Ans => cat4
Ans = x + q;
   Ans => 417
```

Javascript statements

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A statement is a section of JavaScript that can be evaluated by a Web browser
- A script is simply a collection of statements
- It is a good idea to end each program statement with a semi-colon;

Examples:

```
Last_name = “Dunn”;
x = 10 ;
y = x*x ;
```

Type of operator

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A primitive data value is a single simple data value with no additional properties and methods.
- The typeof operator can return one of these primitive types:
- string
- number
- boolean
- undefined
- typeof "John" // Returns "string"
  typeof 3.14 // Returns "number"
  typeof true // Returns "boolean"
  typeof false // Returns "boolean"
  typeof x // Returns "undefined" (if x has no value)

== vs ===

```
// Type conversion is performed before comparison
var v1 = ("5" == 5);        // true
// No implicit type conversion.
// True if only if both types and values are equal
var v2 = ("5" === 5);       // false
var v3 = (5 === 5.0); // true
var v4 = (true == 1); // true (true is converted to 1)
var v5 = (true == 2); // false (true is converted to 1)
var v6 = (true == "1") // true
```

Operators

Arithmetic Operators

+ Addition

- Subtraction

* Multiplication

/ Division

% Modulus

++ Increment

-- Decrement

Relational operators

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Operator	Example	Comments
== (equality)	`'50'==50 // true` `30 ==50 // false`	Returns true when both operands are equal. The operands are converted to the same type before being compared.
!= (non-equality)	`'50'!=50 // false` `30 !=50 // true`	Returns true when both operands are not equal. The operands are converted to the same type before being compared.
=== (equality)	`'50'===50 // false` `50 ===50 // true`	Returns true if both operands are equal and of the same type.
!== (non-	`'50'!==50 // true` `50 !==50 // false`	Returns true if both operands are not equal and of the same type.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

> (greater than)	`'50'>40 // true` `50 >50 // false`	Returns true if the left operand is greater than the right one.
>= (greater than or equal)	`'50'>=50 // true` `30 >=50 // false`	Returns true if the left operand is greater than or equal to the right one.
< (less than)	`'50'<40 // false` `50 <50 // false`	Returns true if the left operand is greater than the right one.
<= (less than or equal)	`'50'<=50 // false` `50 <=50 // true`	Returns true if the left operand is greater than or equal to the right one.

Logical Operator

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Name	Operator	Sample	Description
AND	&&	expession1 && expression2	Both expressions must return true if the outcome shall be true
OR	||	expession1 || expression2	One of the expressions must return true if the outcome shall be true
NOT	!	!expression	If the expression return false the outcome will be true

If else

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Example

```
<script type="text/javascript">
  // Demonstrates if/else and
  // proper use of braces
  // in nested if statements
  var number=4;
  number=prompt("Enter a number less than \n"
    +"5 or greater than 20:");
  if (number >= 5) {
    if (number > 20)
      document.write("Number is greater than 20!<br>");
  }
  else
    document.write("Number is less than 5!<br>");
</script>
```

Conditional operator

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<script type="text/javascript">
  var num1=5, num2=4 ;
  var z = (num1 > num2) ? "num1>num2" : "num1<=num2";
  document.write(z);
</script>
```

Functions

```
function a () { ...
  }

a();
```

Function name

Executes function (aka invokes function)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
var   a   =   function      (){...}
```

Value of function assigned, NOT the returned result!

No name defined

Arguments defined without ‘var’

```
function        compare (x,y) {
    return  x > y;}
```

```
function compare     (x,y){...}

var a          =      compare(4,5);

compare (4,"a");;

compare ();
```

Scope

Global

- Variables and functions defined here are available everywhere

Function

- Variables and functions defined here are available only within this function

Scope Chain

- Everything is executed in an *Execution Context*
- Function invocation creates a new *Execution Context*
  - Each *Execution Context* has:
    - Its own *Variable Environment*
    - Special ‘this’ object
    - Reference to its *Outer Environment*
- Global scope does not have an *Outer Environment* as it’s the most outer

Scope Chain

- Referenced (not defined) variable will be searched for in its current scope first. If not found, the Outer Reference will be searched.
- If not found, the Outer Reference's Outer Reference will be searched, etc.
- This will keep going until the Global scope. If not found in Global scope, the variable is undefined.

Global

```
var x = 2;
```

```
A();
```

Function A

```
var x = 5;
```

```
B();
```

Even though ‘B’ is called within ‘A’

Function B

```
console.log(x);
```

‘B’ is defined within Global

Result: x = 2

DOM

- JavaScript can access and change all the elements of an HTML document
- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can react to all existing HTML events in the page

DOM Merits

- How to change the content of HTML elements
- How to change the style (CSS) of HTML elements
- How to react to HTML DOM events
- How to add and delete HTML elements

HTML DOM

- The HTML elements as objects
- The properties of all HTML elements
- The methods to access all HTML elements
- The events for all HTML elements
- The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

HTML DOM Methods

- HTML DOM methods are actions you can perform (on HTML Elements).
- HTML DOM properties are values (of HTML Elements) that you can set or change.

DOM property

- The easiest way to get the content of an element is by using the innerHTML property.
- The innerHTML property is useful for getting or replacing the content of HTML elements.
- The innerHTML property can be used to get or change any HTML element, including <html> and <body>.

DOM Methods

- The document object represents your web page.
- If you want to access any element in an HTML page, you always start with accessing the document object.

Types of DOM method

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Finding HTML Elements

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find elements by tag name
document.getElementsByClassName(*name*)	Find elements by class name

Change the HTML elements

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Changing HTML Elements

Property	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.style.*property = new style*	Change the style of an HTML element
Method	**Description**
element.setAttribute*(attribute, value)*	Change the attribute value of an HTML element

Finding HTML Element by Id

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<html>
<body>

<h2>Finding HTML Elements by Id</h2>

<p id="intro"></p>
<p>This example demonstrates the <b>getElementsById</b> method.</p>
```

```
<script>
document.getElementById("intro").innerHTML= "How to set the contents for elements using  Elements by Id method";
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<h2>Finding HTML Elements by Id</h2>
<p id="intro">Hello world</p>
<p>This example demonstrates the
<b>getElementsById</b> method.</p>
<h1 id ="change"></h1>
<button onclick="invoke()">ID</button>
```

```
<script>
function invoke()
{
var x =
document.getElementById("intro").innerHTML;
document.getElementById("change").innerHTML= x;
}
</script>
</body>
</html>
```

Finding HTML Elements by Tag Name

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<!DOCTYPE html>
<html>
<body>
<h2>Finding HTML Elements by Tag Name</h2>
<p>Hello World!</p>
<p>This example demonstrates the
<b>getElementsByTagName</b> method.</p>
<p >basics of DOM</p>
<h1 id="demo"></h1>
<button onclick="invoke()"> Change </button>
```

```
<script>
function invoke()
{
var x = document.getElementsByTagName("p");
document.getElementById("demo").innerHTML =
x[2].innerHTML;
}
</script>
</body>
</html>
```

Finding HTML Elements by Class Name

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<!DOCTYPE html>
<html>
<body>
<div class="example">First div element with class="example".</div>
<div class="example">Second div element with class="example".</div>
<p>Click the button to change the text of the first div element with class="example" (index 0).</p>
<button onclick="myFunction()">Try it</button>
<p><strong>Note:</strong> The getElementsByClassName() method is not supported in Internet Explorer 8 and earlier versions.</p>
```

```
<script>
function myFunction() {
    var x = document.getElementsByClassName("example");
    x[0].innerHTML = "Hello World!";
}
</script>


</body>
</html>
```

Finding HTML Elements by Class Name

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<!DOCTYPE html>
<html>
<body>
<h2>Finding HTML Elements by Class Name</h2>
<p>Hello World!</p>
<h1 class="intro">The DOM is very useful.</h1>
<p class="intro">This example demonstrates the <b>getElementsByClassName</b> method.</p>

<h2 class ="intro"> how to use get element by class name</h2>
<h3 id="demo"></h3>
```

```
<script>
var x = document.getElementsByClassName("intro");
document.getElementById("demo").innerHTML = x[0].innerHTML;
</script>
</body>
</html>
```

Changing the Value of an Attribute

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<!DOCTYPE html>
<html>
<body>

<img id="myImage" src="1.jpg"/>
<br/>

<button onclick="invoke()"> Execute </button>
```

```
<script>
function invoke()
{
document.getElementById("myImage").src = "2.jpg";
}
</script>

</body>
</html>
```

Changing the HTML Style

To change the style of an HTML element, use this syntax:
document.getElementById(*id*).style.*property* = *new style*

Changing the HTML Style

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<html>
<body>

<p id="p1">Hello World!</p>
<p id="p2">DOM class</p>
```

```
<script>
document.getElementById("p1").style.color = "red";
document.getElementById("p1").style.backgroundColor = "lime";
document.getElementById("p2").style.fontFamily = "Arial";
document.getElementById("p2").style.fontSize = "75px";
</script>

<p>The paragraph above was changed by a script.</p>

</body>
</html>
```

Event Handling

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Event

Examples of HTML events:

- When a user clicks the mouse
- When a web page has loaded
- When an image has been loaded
- When the mouse moves over an element
- When an input field is changed
- When an HTML form is submitted
- When a user strokes a key

Event Handler Example

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
<form method="POST" action="">
    <input type="button"
        name="myButton"
        value="Click Me"
        onclick="alert('you clicked the button');">
</form>
```

Event Handlers and Event Listeners

- Events are created by activities associated with specific HTML elements.
- The process of connecting an event handler to an event is called **registration**.
- There are two distinct approaches to event handler registration,
  - Assign element attributes
    - Inline event handlers
  - Assign handler addresses to object properties
    - Event handler properties and Event listeners

Approach 1: Assign element attributes (Inline event handlers)

The first button has an inline event handler (onclick attribute) directly in the HTML markup.

When the button is clicked, the handleButtonClickInline function is called, displaying an alert.

Approach 2: Assign handler addresses to object properties

The second button has an id attribute, and its reference is obtained in JavaScript using getElementById.

An event listener is added to the button using addEventListener, linking it to the handleButtonClickListener function.

When the button is clicked, the handleButtonClickListener function is called, displaying an alert.

Event Sources and example events

Events

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Source	Event	Fires When…
Mouse	click	the mouse is clicked and released on an element
	dblclick	an element is clicked twice
	mousemove	every time a mouse pointer moves inside an element
	mouseover	every time a mouse pointer is placed over an element
Keyboard	keydown	when a key is pressed down
	keyup	when a key pressed is released
	keypress	when a key is pressed and released
Form	submit	a form is submitted
	reset	a form reset button is clicked
	focus	an input element is clicked and receives focus
	blur	an input element loses focus

Handson

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Refer->H33 Folder

Event Object Properties

- The event object provides various properties that contain information about the event. Some common properties include:

- type: A string indicating the type of the event (e.g., 'click', 'keydown').
- target: The DOM element that triggered the event.
- currentTarget: The current DOM element that the event is passing through during the capturing or bubbling phase.
- clientX and clientY: The horizontal and vertical coordinates of the mouse pointer when an event occurred.

Event Object Methods:

- The event object also provides methods that can be used to control the event:
- stopPropagation(): Prevents further propagation of the current event in the capturing and bubbling phases.
- preventDefault(): Cancels the event if it is cancelable, meaning that the default action associated with the event will not occur.

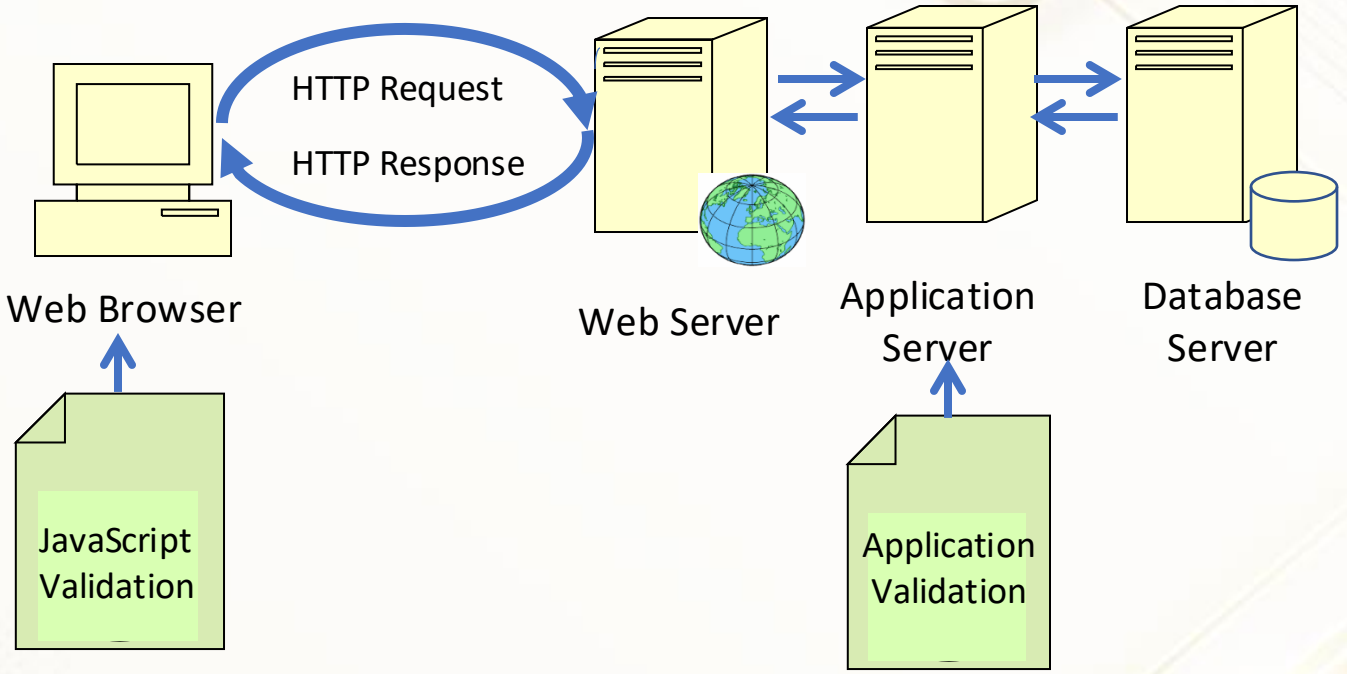
Event Delegation

- Event delegation is a programming pattern in JavaScript where a single event listener is attached to a common ancestor for a group of elements rather than attaching individual event listeners to each element. This is particularly useful when dealing with a large number of dynamically created elements or elements that share a common parent.

JavaScript Form Validation

- Before an HTML form is submitted, JavaScript can be used to provide client-side data validation
- This is more user-friendly for the user than server-side validation because it does not require a server round trip before giving feedback
- If the form is not valid, the formis not submitted until the errors are fixed

Client-Side Validation



- JavaScript data validation happens before form is submitted
- Server-side application validation happens after the form is submitted to the application server

Client-Side vs Server-Side

If creating a Web form, make sure the data submitted is valid and in the correct format

Client-side validation gives the user faster feedback

What to Validate on a Form?

Form data that typically are checked by a JavaScript could be:

- were required fields left empty?
- was a valid e-mail address entered?
- was a valid date entered?
- was text entered in a numeric field?
- were numbers entered in an text field?
- did the number entered have a correct range?